`jwk`	The full key	No	Trusting the sender to give you the key to verify a message loses all security properties.
`jku`	An URL to retrieve the full key	No	The intention of this header is that the recipient can retrieve the key from a HTTPS endpoint, rather than including it directly in the message, to save space. Unfortunately, this has all the issues of the `jwk` header, but additionally opens the recipient up to *SSRF attacks*.

> **DEFINITION** A ***server-side request forgery (SSRF)*** attack occurs when an attacker can cause a server to make outgoing network requests under the attacker's control. As the server is on a trusted network behind a firewall, this allows the attacker to probe and potentially attack machines on the internal network that they could not otherwise access. You'll learn more about SSRF attacks and how to prevent them in chapter 10.

There are also headers for specifying the key as an X.509 certificate (used in TLS). Parsing and validating X.509 certificates is very complex so you should avoid these headers.

6.2.3 Generating standard JWTs

Now that you've seen the basic idea of how a JWT is constructed, you'll switch to using a real JWT library for generating JWTs for the rest of the chapter. It's always better to use a well-tested library for security when one is available. There are many JWT and JOSE libraries for most programming languages, and the https://jwt.io website maintains a list. You should check that the library is actively maintained and that the developers are aware of historical JWT vulnerabilities such as the ones mentioned in this chapter. For this chapter, you can use Nimbus JOSE + JWT